

Université de Ngaoundéré

Master 2

Data Mining, Fouille de données
et Statistiques décisionnelles

Chapitre 2

Classification supervisée et évaluation des modèles

Exercices

Dispensé par

Pr Ndam Njoya Arouna

Chapitre 2 : Travaux dirigés

Thème : classification binaire, métriques, validation et premiers modèles.

Prérequis : chapitre 1 (pipelines, standardisation), exemple `exemples/02_classification_metrics.ipynb` ou `.py`.

Durée indicative : 3 h.

0.1 Partie A : Évaluer un classifieur

0.1.1 Exercice 2.1 : Matrice de confusion et métier anti-spam

Contexte. Classe **positive** = spam. Sur **1000** emails testés : | | Prédit spam | Prédit non-spam

|

1. Reproduisez la matrice et définissez **FP** et **FN** en une phrase chacun (conséquence pour l'utilisateur).
2. Calculez **exactitude**, **précision**, **rappel** et **F1** (formules + valeurs numériques).
3. L'exactitude est-elle une bonne métrique ici ? Argumentez.
4. Si le coût d'un **FN** (spam non détecté) vaut **5 fois** un **FP** (message légitime classé spam), quelle métrique ou objectif privilégieriez-vous pour ajuster le modèle (pas seulement le seuil 0,5) ?

Livrable. Tableau de métriques + 5–8 lignes d'argumentation métier.

0.1.2 Exercice 2.2 : k-NN et effet de k

Jeu. Iris réduit à **2 classes** : setosa (classe 0) vs les deux autres espèces (classe 1). Standardisez les 4 variables **dans un pipeline** avec validation croisée.

Travail.

1. Pour $k \in \{1, 3, 5, 11\}$, estimez **exactitude** et **F1** en **validation croisée stratifiée 5-fold** (`random_state=42`).
2. Remplissez un tableau (4 lignes) et indiquez quel k vous retenez provisoirement.
3. Expliquez pourquoi $k = 1$ est souvent **instable** et pourquoi un k très grand peut **sous-apprendre**.

Livrable. Tableau + graphique optionnel (barres des scores par k). Script : `correction_ex22_knn.py`.

0.2 Partie B : Modèles et sur-apprentissage

0.2.1 Exercice 2.3 : Arbre de décision et profondeur

Sur le **même jeu binaire Iris** : 1. Pour `max_depth` de **1 à 10**, tracez **score moyen CV (5-fold)** et **score sur le train complet** (même profondeur, arbre entraîné sur tout le train du fold ou sur X entier pour la courbe train : précisez votre protocole dans une note).

1. Identifiez la zone de **sur-apprentissage** (écart train CV).
2. Proposez une profondeur **max** raisonnable pour la généralisation et justifiez en 3 lignes.

Livrable. Courbe ou tableau + interprétation. Script : `correction_ex23_arbre.py`.

0.2.2 Exercice 2.4 : Scoring crédit simulé (projet guidé)

Données. `correction_ex24_credit.py` génère `credit_sim.csv` (cible `default`, variables financières).

Étapes obligatoires.

1. **Exploration** : taux de défaut, statistiques descriptives des variables numériques.
2. **Split** train / test (80/20, stratifié sur `default`).
3. **Pipeline** : `StandardScaler` + `LogisticRegression(max_iter=5000)` : fit sur train seulement.
4. **Évaluation** : **AUC-ROC** sur le test ; objectif pédagogique **AUC 0,75**.
5. **Interprétation** : signe et ordre de grandeur des **coefficients standardisés** (2 variables minimum commentées).

Questions.

- Pourquoi la stratification sur une cible déséquilibrée ?
- Citez **une limite** de l'interprétation des coefficients si deux variables sont corrélées.

Livrable. Notebook ou script + réponses. Modèle de code : `correction_ex24_credit.py` (génère les données et entraîne le pipeline). Le notebook `exemples/02_classification_metrics.ipynb` illustre les métriques sur un autre jeu (pas le crédit simulé).

0.3 Partie C : Théorie

0.3.1 Exercice 2.5 : Entropie d'une variable binaire

Soit $Y \in \{0, 1\}$ avec $P(Y = 1) = p$.

1. Écrivez $H(Y)$ en base 2 en fonction de p .
2. Montrez que $H(Y)$ est **maximale** pour $p = 0,5$ (dérivée ou symétrie).
3. Donnez $H(Y)$ pour $p \in \{0,1; 0,9; 0,5\}$ (valeurs numériques).

Livrable. Démonstration courte (8–12 lignes) + tableau des trois entropies.